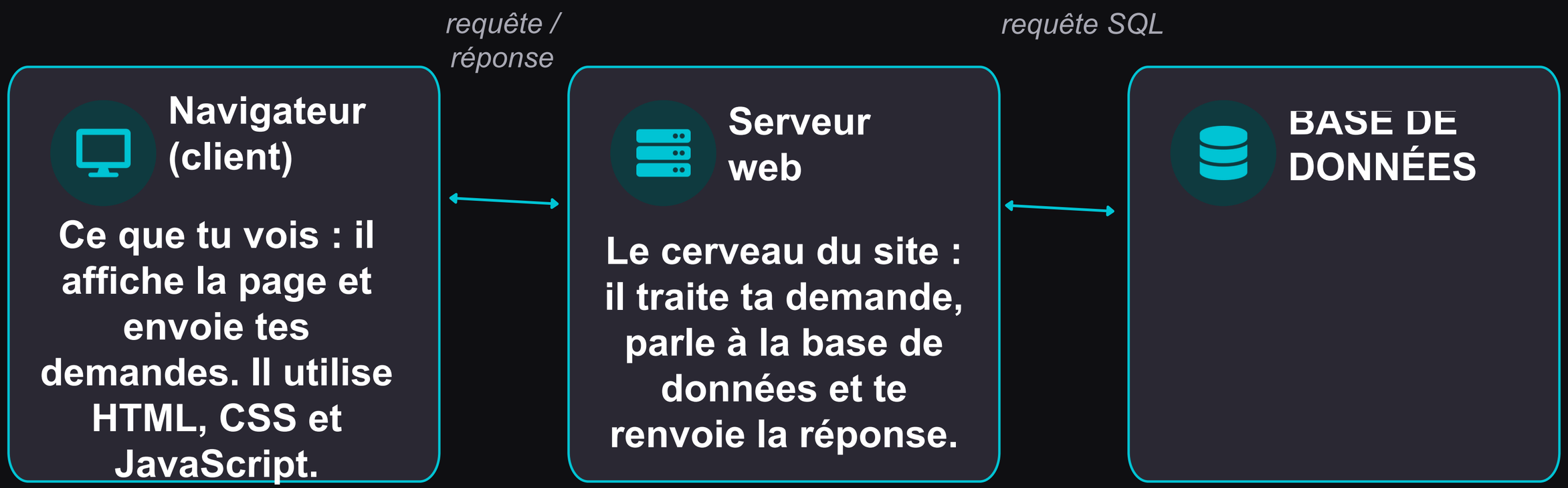
SC2 · SUPCOM CYBERSECURITY CLUB

# WEB EXPLOITATION

# Comment fonctionne un site web ?




✓ **À RETENIR** Le navigateur ne fait qu'afficher ce que le serveur accepte de lui envoyer.

# Une vulnérabilité, c'est quoi ?

## PRÉVU PAR LE DÉVELOPPEUR



### Utilisateur normal

Tape son username et son password → l'application vérifie → accès autorisé ou refusé.

## PAS PRÉVU PAR LE DÉVELOPPEUR



### Utilisateur malveillant

Tape un texte spécial qui change le comportement de l'application → accès sans mot de passe.

✓ **À RETENIR** Une vulnérabilité = une porte laissée ouverte par erreur : le développeur n'y a pas pensé, mais le site l'accepte.

# C'est quoi, l'exploitation web ?

**Trouver une faille dans un site web et s'en servir pour lui faire faire ce que ses créateurs n'avaient pas prévu.**



### Toujours autorisé

Bug bounty, CTF, pentest : toujours avec permission explicite.



### Une méthode

Observer, tester, comprendre. Pas une liste à mémoriser.

✓ **À RETENIR** On s'entraîne uniquement sur des cibles autorisées : le lab de l'atelier.

SC2 · SUPCOM CYBERSECURITY CLUB

# SITE DU SC2



# Ce qu'on a observé ensemble

1

## Login normal

Compte de test : connexion normale, rien de spécial.

2

## Login modifié

On tape un texte spécial dans le champ username.

3

## Accès total

Le site nous laisse entrer, sans mot de passe valide.

✓ **À RETENIR** En quelques secondes, un simple texte a transformé un refus d'accès en accès total.

SC2 · SUPCOM CYBERSECURITY CLUB

# SQL INJECTION (SQLi)

# Base de données + SQL, en bref

**TABLE users**

| id | username | password |
|---|---|---|
| 1 | alice | 1234 |
| 2 | admin | ******** |

Une base de données range les infos dans des tables, comme un tableau Excel.

```
SELECT * FROM users
WHERE username='admin';
```

**SELECT** → quoi récupérer ?
**FROM** → dans quelle table ?
**WHERE** → quelle condition ?

✓ **À RETENIR** SQL = le langage pour parler à une base de données. SELECT / FROM / WHERE suffisent pour aujourd'hui.

# La requête SQL derrière un login

**Formulaire**

Username

Password

Se connecter

**REQUÊTE ENVOYÉE CÔTÉ SERVEUR**

```
SELECT * FROM users
WHERE username = '$username'
AND password = '$password';
```

$username et $password sont remplacés par ce que tu as tapé, tel quel.

✓ **À RETENIR** Sans nettoyage, ce que tape l'utilisateur est collé directement dans le code SQL.

# Où est le problème ?



**Le serveur colle ce que tu tapes dans la requête, sans vérifier.**

Il fait confiance à l'utilisateur. Une injection SQL exploite cette confiance.

✓ **À RETENIR** Entrée utilisateur + requête SQL non filtrée = danger.

# C'est quoi une SQL Injection ?

**Écrire du code SQL dans un champ de saisie non filtré, pour changer le sens de la requête.**

### Ce que le serveur attend

```
alice
```

Un simple nom d'utilisateur.

### Ce qu'un attaquant tape

```
' OR 1=1 --
```

Du code SQL, qui modifie la requête.

✓ **À RETENIR** Le texte tapé n'est plus lu comme un nom, mais comme une partie de la requête.

# Ce que ça permet à un attaquant

### Contourner le login

Se connecter sans connaître aucun mot de passe.

### Lire des données

Extraire identifiants, emails, informations privées.

### Modifier des données

Changer des prix, des soldes, des droits d'accès.

✓ **À RETENIR** Une injection SQL ne « casse » rien : elle fait dire au serveur autre chose que prévu.

SC2 · SUPCOM CYBERSECURITY CLUB

# LOGIN BYPASS

Entrer sans mot de passe

# Comment une entrée modifie la logique

### CE QUE LE DÉVELOPPEUR ÉCRIT

```
SELECT * FROM users
WHERE username='$username' AND password='$password';
```

### CE QUE ÇA DONNE SI username = ' OR 1=1 --

```
SELECT * FROM users
WHERE username='' OR 1=1 --' AND password='...';
```

Le développeur n'a pas voulu cette faille : il a juste collé le texte de l'utilisateur dans la requête.

✓ **À RETENIR** La faille n'est pas voulue : elle vient d'une confiance excessive dans les données reçues.

# Le payload magique, décomposé

| | |
|---|---|
| `'` | → ferme la chaîne de texte SQL |
| `OR` | → ajoute une condition alternative |
| `1=1` | → condition TOUJOURS vraie |
| `--` | → commente (efface) le reste de la requête |

```
' OR 1=1 --
```

# On reconstruit la requête, étape par étape

### 1. Requête originale

```
SELECT * FROM users
WHERE username='$username' AND password='$password';
```

### 2. $username devient ' OR 1=1 --

```
SELECT * FROM users
WHERE username='' OR 1=1 --' AND password='...';
```

### 3. Ce que le moteur SQL lit vraiment

```
SELECT * FROM users
WHERE username='' OR 1=1
```

→ **1=1 est toujours vrai : la base renvoie des comptes, le mot de passe n'est même plus vérifié.**

SC2 · SUPCOM CYBERSECURITY CLUB

# UNION BASED SQLI

Lire les données d'une autre table

# Faire fuiter d'autres données avec UNION

**UNION SELECT colle les résultats d'une 2e requête à la suite de la 1re, même si elle vient d'une autre table.**

**REQUÊTE ORIGINALE**
(page produit)

→

**+ REQUÊTE INJECTÉE**
(table users)

→

**= RÉSULTAT COMBINÉ**
affiché à l'écran

**Seule règle** → même nombre de colonnes dans les 2 requêtes
**Pour le trouver** → ORDER BY 1, 2, 3… jusqu'à l'erreur

✓ **À RETENIR** UNION SELECT = afficher, sur la même page, des données qui viennent d'une autre table.

# SE PROTÉGER

Comment empêcher cette attaque ?

# Requêtes préparées (paramétrées)

✗ MAUVAISE PRATIQUE

```
const query =
  "SELECT * FROM users " +
  "WHERE username='" +
  username + "'";
```

✓ BONNE PRATIQUE

```
const query =
  "SELECT * FROM users " +
  "WHERE username = ?";
db.execute(query, [username]);
```

La requête et les données voyagent séparément : ce que tu tapes reste un simple texte.

✓ **À RETENIR** Avec une requête préparée, l'entrée n'est jamais interprétée comme du code SQL.

# Exemple de filtre

```
function verifierFormulaire(event) {
  const input = document
    .getElementById("username").value;
  const mots = ["OR", "UNION", "SELECT",
                "DROP", "DELETE", "INSERT"];
  for (let mot of mots) {
    if (input.includes(mot)) {
      event.preventDefault(); // bloque
      return false;
    }
  }
  return true; // on envoie
}
```

**Ce que fait ce filtre**

1. Il lit ce que l'utilisateur tape.
2. Il cherche des mots SQL interdits.
3. S'il en trouve : il bloque l'envoi.

✓ **À RETENIR** Ça a l'air efficace… mais est-ce suffisant ? Slide suivante.

# VALIDATION FRONTEND UNIQUEMENT

# Pourquoi un simple filtre ne suffit pas

***« Je bloque juste le caractère ' et voilà, c'est sécurisé. »***

### Trop de variantes

Majuscules, encodages, autres syntaxes : un filtre ne couvre jamais tout.

### Ça bloque des gens honnêtes

Un nom comme O'Brien contient un ' tout à fait normal.

### Ce n'est pas la vraie cause

Le vrai problème, c'est de coller le texte dans la requête. Le filtre cache le symptôme.

✓ **À RETENIR** Un filtre est un pansement. Les requêtes préparées traitent la cause.

# Frontend vs backend : la différence

**FRONTEND (navigateur)**



## JavaScript dans le navigateur

Vérifie qu'un champ n'est pas vide, qu'un email est valide… juste pour le confort de l'utilisateur.

**BACKEND (serveur)**



## Code exécuté sur le serveur

La seule vérification que l'utilisateur ne peut ni voir ni modifier : la seule fiable.

✓ **À RETENIR** Le frontend, c'est toi qui l'exécutes dans ton navigateur : tu peux toujours le désactiver ou le contourner.

# Rappel : le filtre tourne dans le navigateur

## LE FILTRE JAVASCRIPT (CÔTÉ FRONTEND)

```
const motsInterdits = ["OR", "UNION", "SELECT", "--"];
if (motsInterdits.some(m => username.includes(m))) {
  alert("Mot-clé SQL interdit !");
  return; // on bloque l'envoi du formulaire
```

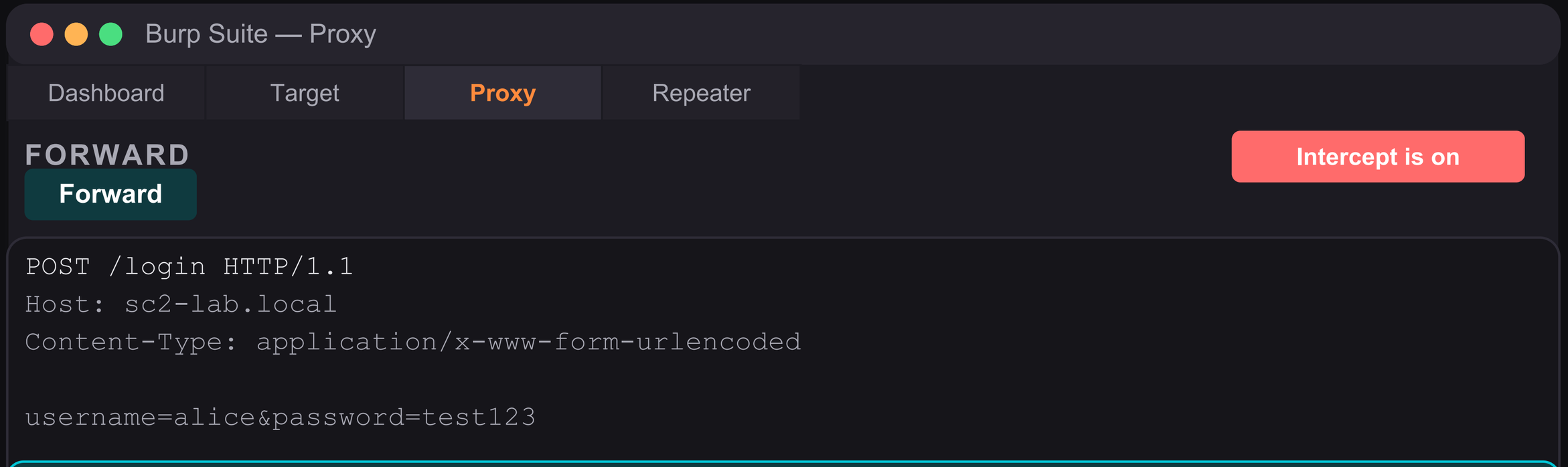
D'abord, un petit rappel sur SQL

**Ce filtre tourne dans ton navigateur. Donc toi, l'utilisateur, tu le contrôles entièrement.**

✓ **À RETENIR** Prochaine étape : on contourne ce filtre avec Burp Suite, un outil qui intercepte les requêtes.

# Contourner avec Burp Suite — intercepter

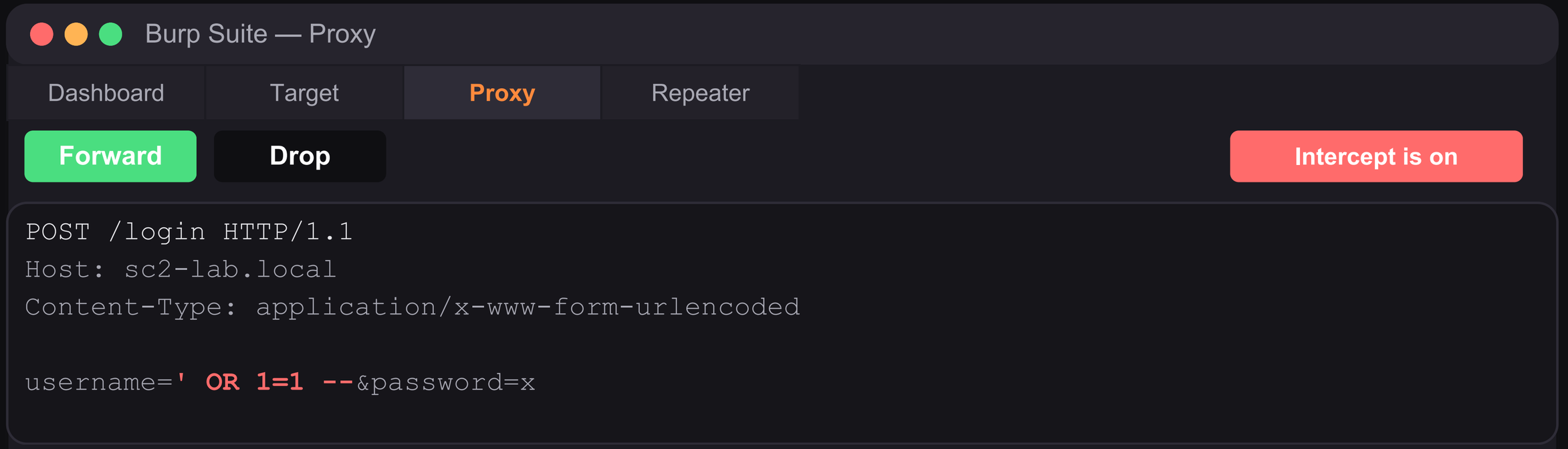
Burp Suite se place entre le navigateur et le serveur : il peut intercepter et modifier chaque requête avant qu'elle parte.




✓ **À RETENIR** La requête capturée vient directement du navigateur — avant qu'elle n'atteigne le serveur.

# Contourner avec Burp Suite — modifier

Le filtre JavaScript ne s'exécute plus : on modifie directement le texte brut de la requête, avant qu'elle parte vers le serveur.




✓ **À RETENIR** On tape le payload directement dans Burp, pas dans le formulaire : le filtre JavaScript est totalement court-circuité.

# Résultat : accès obtenu malgré le filtre

Burp Suite — Response

Pretty | Raw | Hex | Render

```
HTTP/1.1 200 OK
Content-Type: text/html
```

✓ **À RETENIR** Le filtre bloquait le formulaire, pas la requête. Côté serveur, rien n'a vérifié ni filtré l'entrée : c'est la vraie faille.

# La règle qui ne souffre aucune exception



**Ne jamais faire confiance aux données venant du client.**

Tous les contrôles importants (prix, droits, quantités, permissions) doivent être re-vérifiés côté serveur.

✓ **À RETENIR** Le frontend améliore l'expérience utilisateur. Le backend garantit la sécurité.

# Testez vos connaissances (1/2)

**Q1 · Qu'est-ce qu'une vulnérabilité ?**
✓ Une faille, une porte laissée ouverte par erreur dans le code d'un site web.

**Q2 · Qu'est-ce que l'exploitation web ?**
✓ Utiliser cette faille pour faire faire au site ce qu'il ne devrait pas (ex : afficher des données secrètes).

**Q3 · Comment fonctionne le web ?**
✓ Navigateur → serveur → base de données.

**Q4 · En une phrase, c'est quoi une SQL Injection ?**
✓ Taper du code SQL dans un champ de formulaire pour piéger la base de données.

# Testez vos connaissances (2/2)

**Q5 · Pourquoi ' OR 1=1 -- contourne un login ?**
✓ La condition est toujours vraie et le mot de passe est mis en commentaire.

**Q6 · C'est quoi un UNION SELECT ?**
✓ Il fusionne les résultats de deux requêtes en un seul tableau, même de tables différentes.

**Q7 · Pourquoi utiliser des requêtes préparées ?**
✓ L'entrée utilisateur n'est jamais interprétée comme du code SQL.

**Q8 · Frontend ou backend : lequel est fiable ?**
✓ Le frontend se contourne ; seule la validation serveur est fiable.

# MERCI POUR VOTRE ATTENTION